UNIVERSIDAD TECNOLÓGICA NACIONAL — FACULTAD REGIONAL SAN NICOLÁS

Seguimiento de Gastos Personales

CÁTEDRA	ARQUITECTURA	GESTIÓN DE ESTADO	FORMULARIOS Y VALIDACIÓN
Programación IV (Práctica 4)	Single Page Application (SPA)	React Context API	React Hook Form + Zod

1. Justificación Técnica y Objetivos Pedagógicos

El objetivo central de este desarrollo es estructurar una solución web declarativa, modular y mantenible que resuelva la carga, modificación, eliminación y análisis porcentual de egresos económicos individuales. La aplicación supera los límites del paso tradicional de propiedades (*prop drilling*), implementando una única fuente de la verdad reactiva mediante **Context API** y garantizando tipado en tiempo de ejecución a través de esquemas formales en **Zod**.

2. Diagrama Arquitectónico del Flujo de Datos



El `GastosProvider` envuelve el árbol en `App.jsx`. Las páginas consumen el estado mediante el hook de conveniencia `useGastos()`. Esto asegura que cualquier mutación (creación, edición o baja) actualice instantáneamente a los observadores sin acoplamiento jerárquico.

3. Centralización de Estado y Mutaciones Inmutables

```
// src/context/GastosContext.jsx const agregarGasto = (datos) => { const nuevo = { id: nextId, gasto: datos.gasto.trim(), importe: Number(datos.importe), rubro: datos.rubro }; setGastos((prev) => [nuevo, ...prev]); // Inserción al inicio (cronológico descendente) setUltimoId(nextId); setNextId((prev) => prev + 1); }; const eliminarGasto = (id) => { setGastos((prev) => prev.filter((g) => g.id !== Number(id))); // Number() previene errores con string params setUltimoId((prev) => (prev === Number(id) ? null : prev)); };
```

Inmutabilidad y Conciliación: React detecta cambios verificando identidades de memoria. Se emplea sintaxis spread (`[nuevo, ...prev]`) y `.filter()` para retornar instancias nuevas. Adicionalmente, desacoplar `setGastos` de `setNextId` previene colisiones generadas por el doble disparo preventivo de *React StrictMode* en desarrollo.

4. Validación Estricta con Zod y Tipado Fuerte

Para erradicar validaciones imperativas con bloques `if` dispersos, se diseñó un esquema centralizado que actúa como contrato formal. Se exporta desde el contexto para asegurar que tanto el alta como la edición compartan exactamente el mismo criterio de integridad de datos.

```
export const gastoSchema = z.object({ gasto: z.string().trim().min(1, 'El gasto es obligatorio'), importe: z.coerce.number({ invalid_type_error: 'Debe ingresar un número' }).positive('Debe ser mayor a 0'), rubro: z.string().min(1, 'Debe seleccionar un rubro'), });
```

El método `z.coerce.number()` resulta crítico: los inputs nativos HTML siempre emiten valores de tipo `string`. La coerción realiza el parseo implícito a primitivo numérico antes de ejecutar la aserción de positividad (`.positive()`), previniendo desfasajes de tipado.

5. React Hook Form y la Carrera de Renderizado (El Rol de watch)

React Hook Form opera mediante **componentes no controlados**: almacena referencias nativas (`ref`) y delega la gestión de caracteres al DOM del navegador para evitar ciclos continuos de re-renderizado. No obstante, al coexistir con `GastosContext` y el estado local de alerta, se produce un escenario de concurrencia:

> **Causa técnica del bloqueo de inputs resueltos mediante `watch()`:**
> En `onSubmit` se disparan tres operaciones: `agregarGasto()` (mutación en Context), `setAlerta(true)` (mutación en estado local) y `reset()` (limpieza nativa por ref). Gracias al *Automatic Batching* de React 18, los cambios de estado se agrupan en un único repintado. El DOM virtual reconstruye el árbol pero no reconcilia los valores visuales de inputs no controlados, reteniendo el texto previo. Al ejecutar **`watch()`** dentro del cuerpo del componente, se activa un listener reactivo interno que fuerza a la librería a sincronizar el repintado de los campos con los valores vacíos del reset.

Mecanismo	Comportamiento en Render	Impacto en UI y Ciclo de Vida
Sin watch()	No controlado puro (0 re-renders en escritura).	Riesgo de desincronización tras `reset()` con batching de React 18.
Con watch() activo	Suscripción reactiva a cambios del form.	Garantiza limpieza visual inmediata de inputs al enviar exitosamente.
useState tradicional	Controlado (1 re-render por pulsación).	Genera latencia innecesaria y colisiona con `register()` de RHF.

6. Notificación Flotante y Resaltado Visual Condicional

En estricto cumplimiento de la consigna gráfica, el cartel de confirmación verde ✔ `Ingresado.` se desacopla del flujo del documento mediante `position: fixed; top: 18px; left: 50%; transform: translateX(-50%);` con `z-index: 9999`. Por su parte, la tabla de gastos identifica unívocamente la última carga mediante la clase `fila-resaltada` aplicada exclusivamente si `g.id === ultimoId`.

7. Enrutamiento SPA y Ocultamiento de Navegación

La especificación formal exige: *"esta edición se debe mostrar ocupando la totalidad del formulario y la navegación"*. Para resolver esto a nivel arquitectura sin alterar el árbol de rutas en App.jsx, se integró el hook useLocation:

```
// src/App.jsx function ContenidoApp() { const location = useLocation(); const editando = location.pathname.startsWith('/modificar'); return ( <> {!editando && <nav className="navbar-gastos">...</nav>} <main className="main-content"> <Routes> <Route path="/" element={<Gastos />} /> <Route path="/modificar/:id" element={<ModificarGasto />} /> <Route path="/resumen" element={<Resumen />} /> </Routes> </main> </> ); }
```

Cuando el usuario navega a /modificar/:id, la expresión !editando evalúa a falso, desmontando el menú de cabecera y cediendo el ancho y alto del viewport a la interfaz de edición.

8. Pantalla de Modificación Reactiva sin useEffect

En lugar de usar un useEffect manual para setear valores tras montar la pantalla, se aprovechó la propiedad nativa values de React Hook Form. Al pasarle directamente la referencia del gasto encontrado (values: gastoAModificar), la librería puebla los inputs de manera reactiva e instantánea.

Controles Obligatorios en ModificarGasto:

1. **Enviar:** Actualiza el objeto en el contexto y redirige con navigate('/').
2. **Eliminar:** Ejecuta eliminarGasto(id) con type="button" obligatorio para evitar disparar la validación de submit.
3. **Ícono de Retorno:** Enlace <Link to="/">↩️</Link> que aborta la edición y regresa a la tabla inicial.

9. Algoritmo de Consolidación y Métricas en Resumen

La vista de métricas procesa el listado plano de gastos en tiempo lineal mediante reduce() y acumulación de diccionarios. El porcentaje de participación económica se calcula en tiempo de ejecución:

Porcentaje por Rubro (%) = (Total del Rubro ÷ Total General) × 100

Rubro	Descripción	Método de Agrupación
ALM / COM	Almacén / Combustible	Acumulador dinámico en objeto JavaScript: acc[rubro] = (acc[rubro] || 0) + importe
BEB / SAL / LIM	Bebidas / Salidas / Limpieza	Conversión a matriz con Object.entries() para renderizar la tabla con totales formateados.

10. Banco de Preguntas y Respuestas para la Exposición Oral

Guía preparada para responder con precisión terminológica y solvencia técnica ante las preguntas más habituales de los docentes evaluadores de Programación IV:

Q1: ¿Por qué se optó por Context API frente a Redux o prop drilling?
Respuesta: Prop drilling hubiera obligado a ensuciar `App.jsx` con múltiples estados y callbacks cruzados hacia rutas independientes. Para el alcance de esta aplicación, Redux introducía un sobrecosto de boilerplate innecesario. Context API resuelve de forma nativa la distribución global del estado con una única fuente de la verdad reactiva.

Q2: ¿Por qué durante las pruebas se insertaban gastos duplicados al enviar?
Respuesta: Se debió a la interacción con `React.StrictMode` en desarrollo, el cual ejecuta las funciones actualizadoras de estado `(prev) => ...` dos veces intencionalmente para detectar efectos colaterales impuros. Al tener llamadas de mutación anidadas dentro de un setter, la adición se replicaba. Se resolvió desacoplando cada setter de forma atómica.

Q3: ¿Por qué saltaban errores de Zod si había texto visible en los inputs?
Respuesta: Ocurría por conflicto entre inputs controlados (con `value` y onChange propios) y no controlados. El estado local de React retenía el texto visualmente, pero pisaba las referencias internas de React Hook Form. Al ejecutar `handleSubmit`, la librería evaluaba campos vacíos contra Zod. La solución fue dejar el control absoluto a `{...register()}`.

Q4: ¿Por qué el botón "Eliminar" en ModificarGasto debe llevar type="button"?
Respuesta: Dentro de una etiqueta `<form>`, cualquier botón sin atributo explícito adopta por defecto el comportamiento `type="submit"`. Si no se declara `type="button"`, al querer eliminar se disparaba la validación de Zod y el submit del formulario en lugar de la función de borrado directo.

Q5: ¿Por qué es fundamental Number(id) al eliminar o modificar en el Contexto?
Respuesta: El hook `useParams()` de React Router extrae todos los parámetros de la URL como cadenas de texto (ej. `"3"`), mientras que en el estado los identificadores son números enteros (3). Una comparación estricta `3 !== "3"` evalúa siempre a verdadero, impidiendo que el método `.filter()` excluya el registro seleccionado.

Q6: ¿Cómo se garantiza que el último gasto sea el resaltado y esté arriba?
Respuesta: Al insertar un nuevo registro se aplica `setGastos((prev) => [nuevo, ...prev])`, ubicándolo en la primera posición del array. Paralelamente, se almacena su identificador en el estado `ultimoId` del contexto. Al renderizar la tabla, cada fila verifica si `g.id === ultimoId` para asignar dinámicamente la clase CSS celeste.